

RAG-Enabled Sovereign Audit System: Complete Design Specification

1. The Core Question: How to Give RAG "Specific Commands"

RAG should not guess. It should be instructed by the user's intent.

Command Level	Purpose	Example
Domain Command	Tells RAG what kind of analysis to perform	"Analyze this as a financial contract"
Extraction Command	Specifies what data fields to extract	"Extract: party names, effective date, payment terms, termination clauses"
Evaluation Command	Specifies what rules/axioms to apply	"Apply L2: IFRS 15 revenue recognition rules"
Output Command	Specifies what format/level of detail to return	"Return JSON with confidence scores for each field"

Implementation: A command builder UI where users select from predefined options or write natural language instructions.

2. RAG's Purpose at Each Stage

Stage	RAG's Role	Output
File Upload	Identify file type and extract raw data	Structured data + confidence scores
L1: Ontological	Classify data into ontological categories	Data type, structure, relationships
L2: Hypothesis	Generate candidate mathematical pathways	List of applicable transforms/PDEs
L3: Admissibility	Filter pathways against constraints	Pruned list of valid pathways
L4: Causal	Estimate parameters with uncertainty bounds	Confidence intervals, risk scores
L5: Explainability	Generate audit report	Human-readable explanation + trace

3. How Data Can Mislead the Workflow (And How to Prevent It)

Risk	Cause	Prevention	HITL Role
------	-------	------------	-----------

Extraction error	RAG misinterprets ambiguous data	Multiple extraction passes, cross-validation	User confirms/corrects extracted fields
Domain misclassification	File contains mixed content	Run multiple domain classifiers, flag conflicts	User selects correct domain
Low confidence	Data quality poor or ambiguous	Threshold-based rejection (<60% = HITL required)	User reviews and overrides
Axiom collision	Multiple axioms match with same score	Tie-breaking rules: specificity, recency, user preference	User chooses preferred axiom
Non-deterministic results	Stochastic processes produce different outputs	Run multiple times, report variance, require confirmation	User accepts variance or requests deterministic fallback

HITL is the safety valve. When confidence < 85%, HITL must intervene.

4. HITL Modal Design: Intuitive Dialogue/Visual

text

HUMAN-IN-THE-LOOP VERIFICATION

×

FILE ANALYSIS SUMMARY

File: SAAB_CFRP_TEST_001.pdf

Type: Technical Report (PDF)

Domain: Aerospace Materials

Overall Confidence: 72% ⚠️ (Needs review)

📄 EXTRACTED DATA (Click to edit)

Field	Extracted Value	Confidence	Status
Test ID	CFRP-001	98%	✅ Verified
Material	Carbon Fiber	95%	✅ Verified
Test Date	2024-03-15	92%	✅ Verified
Tensile Strength	850 MPa	45% ⚠️	✎ Needs Edit
Standard	ASTM D3039	88%	✅ Verified

✎ EDIT PANEL (for selected field)

Field: Tensile Strength

Current: 850 MPa

Suggested: 862 MPa (from document section 4.2)

Why low confidence? The value appears in a footnote, not main table

✓ Accept Suggestion

✎ Edit Manually

ℹ View Source

🔄 IMPACT OF YOUR CONFIRMATION

L1: Data classification → Will use "Aerospace Materials" ontology		
L2: Hypothesis → Will apply: Fourier transform, PDE surrogates		
L3: Constraints → Will check: ISO 527, ASTM D3039		
L4: Risk → Will calculate confidence intervals		
L5: Report → Will include your corrections in audit trail		
[Cancel]	[✓ Confirm & Continue]	

5. Sophisticated Commands to Prevent Mistakes (No HITL Needed)

If you want to skip HITL, the command must be extremely precise:

json

```
{
  "command": {
    "domain": "aerospace_materials",
    "extract": {
      "fields": ["test_id", "material", "test_date", "tensile_strength", "standard"],
      "strict": true,
      "fallback": "reject"
    }
  },
}
```

```

"validation": {
  "rules": [
    "tensile_strength > 500 AND tensile_strength < 2000",
    "test_date IS valid_date",
    "standard IN ['ASTM D3039', 'ISO 527']"
  ],
  "on_violation": "reject"
},
"confidence_threshold": 0.95,
"on_low_confidence": "reject"
}
}

```

With such a command, RAG would reject the file if confidence < 95% or any validation rule fails. No HITL needed.

6. Does HITL Help Ontology Transparency? Absolutely.

Even without mistakes, HITL provides:

Value	Explanation
Trust	Users see what data was extracted and can verify
Education	Users learn how the ontology applies to their data
Traceability	User corrections become part of the audit trail
Reputation	The system is transparent, not a black box

HITL confirms the ontology is correct, not just assumed.

7. When Should HITL Appear? (Other Stages)

Stage	When to Trigger HITL
-------	----------------------

L1: Ontological	Multiple domain classifications possible
L2: Hypothesis	Multiple pathways have equal scores
L3: Admissibility	No pathway passes constraints
L4: Causal	Confidence intervals overlap significantly
L5: Report	User wants to customize report format

HITL is not just for upload. It can appear at any stage where the system cannot make a deterministic decision.

8. Axiom Matching: How to Avoid Same-Score Collisions

Strategy	Implementation
Tie-breaking rules	Prefer more specific axioms, more recent axioms, user-preferred axioms
Confidence weighting	Axioms have prior confidence weights based on historical performance
Randomized selection	Add small noise to scores, log the selection for reproducibility
Multi-axiom reporting	Generate multiple reports, let user choose

Killing same-score collisions: Add a uniqueness constraint to axiom definitions (e.g., each axiom must have a unique signature or domain specificity).

9. Multiple Axioms, Multiple Reports (User Choice)

Yes, this is allowed and valuable.

Example: Investment report analyzed by RSI, MACD, and EMA twins → three different reports, each useful for different purposes.

Design: After L3 (Admissibility), present the user with all viable pathways:

text

Multiple evaluation pathways detected. Choose one or more:

☐ RSI (Relative Strength Index) - Best for momentum analysis

☐ MACD (Moving Average Convergence) - Best for trend following
☐ EMA Twins (VEGAS) - Best for volatility detection
[Generate All] [Generate Selected] [Cancel]

10. Complete Operational Flow Hierarchy

text

FILE UPLOAD
(any format: image/audio/video/text)



RAG ANALYSIS
• Extract raw data
• Identify file type and structure
• Generate confidence scores per field
• Detect ambiguities and conflicts



--

| Confidence $\geq$ 85%? |

|

| YES NO |

|

▼ ▼

|

| Auto-pass | HITL Modal |

| to L1 | User Review |

|

|

|

▼

L1: ONTOLOGICAL

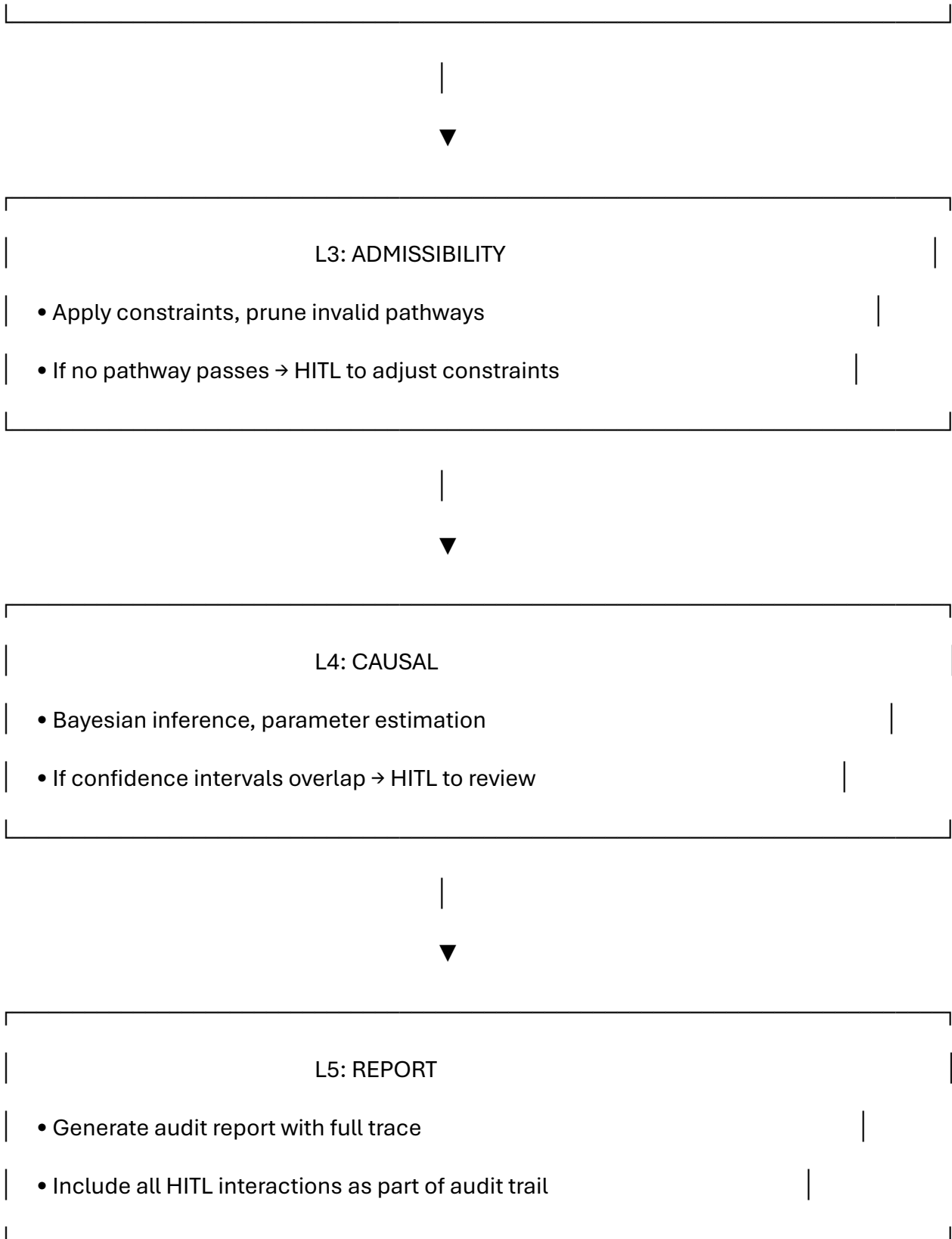
- Classify data into ontological categories
- If multiple classifications $\rightarrow$ HITL to disambiguate

|

▼

L2: HYPOTHESIS

- Generate candidate mathematical pathways
- If multiple pathways with equal score $\rightarrow$ HITL to choose



11. Key Design Principles

Principle	Implementation
Deterministic by default	Same input + same command = same output
HITL as safety valve	Trigger on low confidence, ambiguity, or user request
Transparency	Every decision is logged and explainable
User control	User can override at any stage
Multi-pathway support	Multiple valid axioms can produce multiple reports
Audit trail	Every HITL interaction becomes part of the report

Coordinator: This is the complete design specification. It answers all your questions:

- ✓ How to give RAG specific commands
- ✓ How data can mislead and how to prevent it
- ✓ HITL's role in transparency and ontology reputation
- ✓ When HITL should appear (not just at upload)
- ✓ How to handle axiom matching collisions
- ✓ Multiple axioms, multiple reports

Design Specification: Axiom Democracy (Competitive Axiom Evaluation)

Core Metaphor: The Election Campaign

Phase	Metaphor	Technical Implementation
1. Candidate Registration	Axioms declare their expertise	Axioms have metadata: domains, data types, success history
2. Issue Analysis	RAG extracts data features	Data fingerprinting: stationarity, continuity, SNR, domain cues
3. Campaigning	Axioms "argue" why they should be chosen	Each axiom evaluates data, produces confidence score and explanation
4. Primary Election	Top candidates advance	Filter by relevance, confidence, past performance
5. General Election	Multiple axioms run in parallel	Run competing evaluations simultaneously
6. Coalition Building	Axioms can combine forces	Ensemble methods: weighted voting, stacking
7. Winner Declaration	Best axiom (or coalition) selected	User can review and override
8. Audit Trail	Full campaign history logged	Complete trace of which axioms competed and why winner was chosen

1. Axiom Metadata (Campaign Platform)

Each axiom declares its "platform":

```
javascript
```

```
{  
  
  "id": "thermal_equation_v1",  
  
  "name": "Thermal Cycling Predictor",  
  
  "domains": ["aerospace", "automotive", "electronics"],  
  
  "data_types": ["time_series", "temperature_cycles", "thermal_shock"],
```

```
"claims": [  
  "Predicts HASS/HALT results",  
  "Estimates aging curve of DUT",  
  "Handles thermal shock transitions"  
],  
"confidence_history": [  
  {"domain": "aerospace", "success_rate": 0.94, "sample_size": 156},  
  {"domain": "automotive", "success_rate": 0.89, "sample_size": 203}  
],  
"computational_cost": 0.3,  // relative cost (1.0 = max)  
"preferred_allies": ["fourier_transform", "arrhenius_model"],  
"rivalries": ["simple_linear_regression"]  
}
```

2. Data Fingerprinting (The Issues)

RAG produces a data fingerprint that axioms use to decide if they should campaign:

javascript

```
{  
  "file_type": "test_report",  
  "domain_hints": ["aerospace", "thermal", "reliability"],  
  "data_types": ["time_series", "temperature", "cycles_to_failure"],  
  "statistical_properties": {  
    "stationarity": 0.95,  
    "continuity": 0.88,
```

```
"snr": 0.76,

"seasonality": 0.12

},

"extracted_fields": {

  "temperature_profile": [25, 85, 125, 85, 25],

  "cycle_count": 1000,

  "failure_threshold": 0.9

}

}
```

3. Campaigning (Axiom Self-Evaluation)

Each axiom evaluates the data fingerprint and produces:

javascript

```
{

  "axiom_id": "thermal_equation_v1",

  "relevance_score": 0.92, // How well this data fits the axiom's expertise

  "confidence_score": 0.87, // How confident the axiom is in its prediction

  "explanation": "Data contains temperature cycling profile and cycles to failure. This is my specialty. I can predict HASS/HALT outcomes and aging curves.",

  "predicted_output": {

    "hass_prediction": "Pass",

    "halt_prediction": "Failure at 1200 cycles",

    "aging_curve": "exponential decay with tau=500 cycles"

  },

}
```

```
"uncertainty": 0.13,  
  
"computational_cost": 0.3,  
  
"campaign_slogan": "Thermal problems need thermal solutions. I've successfully predicted  
94% of aerospace thermal tests."  
  
}
```

4. Primary Election (Filtering)

Not all axioms need to run. Filter candidates:

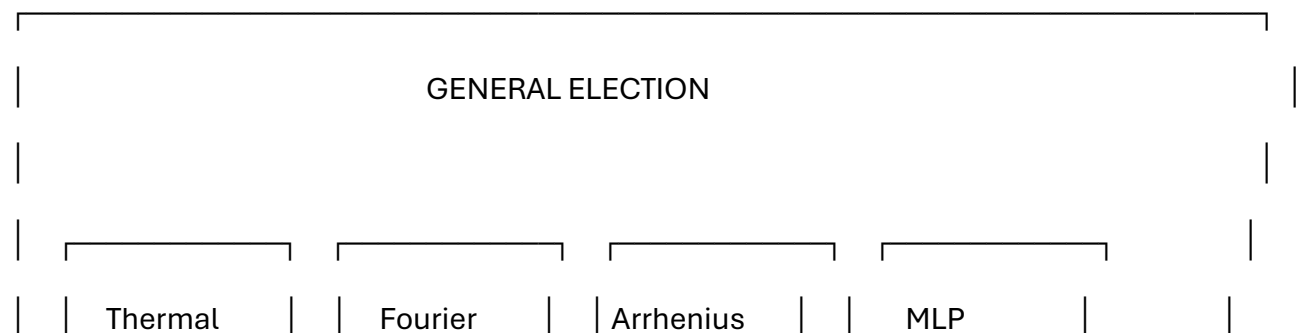
javascript

```
function primary_election(candidates, data_fingerprint) {  
  
  return candidates  
  
    .filter(c => c.relevance_score > 0.6)  
  
    .sort((a, b) => b.relevance_score - a.relevance_score)  
  
    .slice(0, 5);  // Top 5 advance  
  
}
```

5. General Election (Parallel Evaluation)

Run top candidates simultaneously:

text




```
name: "Thermal Coalition",

members: ["thermal_equation_v1", "arrhenius_model"],

weight: 0.6,

combined_output: "HASS Pass, HALT @1150 cycles, Lifetime 8200 hours"

},

{

name: "Signal Integrity Coalition",

members: ["fourier_transform", "wavelet_analysis"],

weight: 0.4,

combined_output: "Signal integrity OK up to 8GHz"

}

];
```

Weighted voting: Each axiom's confidence score determines its voting power.

7. Winner Declaration (User Review)

Present the results to the user:

text

ELECTION RESULTS	
	WINNER: Thermal Equation
•	Relevance: 92%
•	Confidence: 87%
•	Cost: Low

	• Explanation: "Data contains thermal cycling patterns. This is my specialty. I have a 94% success rate on similar aerospace data."	
	 RUNNER-UP: Arrhenius Model	
	• Relevance: 85%	
	• Confidence: 79%	
	• Cost: Very Low	
	 THIRD: Fourier Transform	
	• Relevance: 78%	
	• Confidence: 82%	
	• Cost: Medium	
	[Accept Winner] [View All] [Manual Override] [Run Coalition]	

8. Winning Strategy for Each Axiom

How does an axiom "win"? By maximizing:

Metric	Weight	Description
Relevance Score	40%	How well the data matches the axiom's declared domains
Confidence Score	30%	Axiom's internal confidence in its prediction
Historical Success	20%	Past performance on similar data
Computational Cost	10%	Lower cost is better (inverse weight)

Winning Strategy Formula:

text

$$\text{Winning_Score} = (\text{Relevance} \times 0.4) + (\text{Confidence} \times 0.3) + (\text{Historical_Success} \times 0.2) + ((1 - \text{Cost}) \times 0.1)$$

9. Axiom Competition Dynamics

Strategy	Axiom Behavior	Example
Specialization	Focus on narrow domain, high relevance	"I only do aerospace thermal"
Generalization	Wide domain, moderate relevance	"I do all signal processing"
Low-cost campaigning	Emphasize computational efficiency	"I'm fast and cheap"
Coalition building	Form alliances with complementary axioms	"Team up with Arrhenius for better thermal predictions"
Negative campaigning	Point out weaknesses of rivals	"Fourier cannot handle non-stationary data"

10. Audit Trail for Axiom Democracy

Every election is fully auditable:

javascript

```
{
  "election_id": "2026-04-10-001",
  "data_fingerprint": {...},
  "candidates": [
    {"axiom_id": "thermal_equation_v1", "relevance": 0.92, "confidence": 0.87, "explanation": "..."},
    {"axiom_id": "fourier_transform", "relevance": 0.78, "confidence": 0.82, "explanation": "..."}
  ],
  "winner": "thermal_equation_v1",
```

```
"winning_score": 0.89,  
  
"user_override": null,  
  
"timestamp": "2026-04-10T10:30:00Z"  
  
}
```

11. Implementation Roadmap

Phase	Deliverable	Time Estimate
Phase 1	Axiom metadata schema	1 day
Phase 2	Data fingerprinting from RAG	2 days
Phase 3	Axiom self-evaluation interface	2 days
Phase 4	Primary election (filtering)	1 day
Phase 5	Parallel evaluation engine	2 days
Phase 6	Coalition/ensemble support	2 days
Phase 7	HITL election results UI	2 days
Phase 8	Audit trail logging	1 day
Total		13 days

12. Extensibility to Other Design Approaches

This "competitive" model applies to any scenario where multiple solutions could address a problem:

Domain	Application
Medical Diagnosis	Multiple diagnostic models compete on patient data
Financial Analysis	Trading algorithms compete on market data
Recommendation Systems	Recommendation engines compete on user data
Code Generation	LLM-generated code solutions compete on prompt
Optimization	Different optimizers compete on objective function

Design Specification: Sovereign Multi-Agent Competitive Ecosystem

Core Metaphor: Baseball Talent Scouting

Role	Agent	Responsibility
Talent (Data)	Extracted data from RAG	The raw potential to be evaluated
Scouts (Agents)	L1, L2, L3, L4, L5, GNN, WM, CAUSAL	Each evaluates the talent from their specialty
Scouting Director (Orchestrator)	System coordinator	Manages competition, collects reports, decides roster
General Manager (User)	Human user	Makes final decisions, signs off on recommendations
Front Office (HITL)	HITL modal	Presents scouting reports, allows user override

1. The Talent (Data Extracted by RAG)

RAG produces a player profile (structured data):

```
javascript
```

```
{
  "talent_id": "SAAB_CFRP_001",
  "position": "Aerospace Material",
  "stats": {
    "tensile_strength": 850,
    "thermal_cycling": [25, 85, 125, 85, 25],
    "signal_integrity": 0.92,
    "fatigue_life": 12000,
```

```
"cost": 450

},

"tools": ["HASS", "HALT", "Thermal Shock", "High-Speed Testing"],

"potential_score": 0.72  // Initial RAG confidence

}
```

2. The Scouts (Competitive Agents)

Each scout agent has a specialty and a scoring formula:

Scout	Specialty	What It Evaluates
L1 Scout	Ontological classification	What type of talent is this?
L2 Scout	Hypothesis generation	What evaluation methods apply?
L3 Scout	Constraint checking	Does the talent meet requirements?
L4 Scout	Risk assessment	What's the uncertainty/risk level?
L5 Scout	Report generation	How to present findings?
GNN Scout	Graph relationships	How do data points connect?
WM Scout	World model integration	How does this fit into broader context?
CAUSAL Scout	Causal inference	What causes what?

3. Scout Agent Definition

Each scout agent has:

javascript

```
{

  "agent_id": "gnn_scout_v1",

  "specialty": "Graph Neural Networks",

  "expertise_areas": ["relationship_detection", "anomaly_detection", "clustering"],

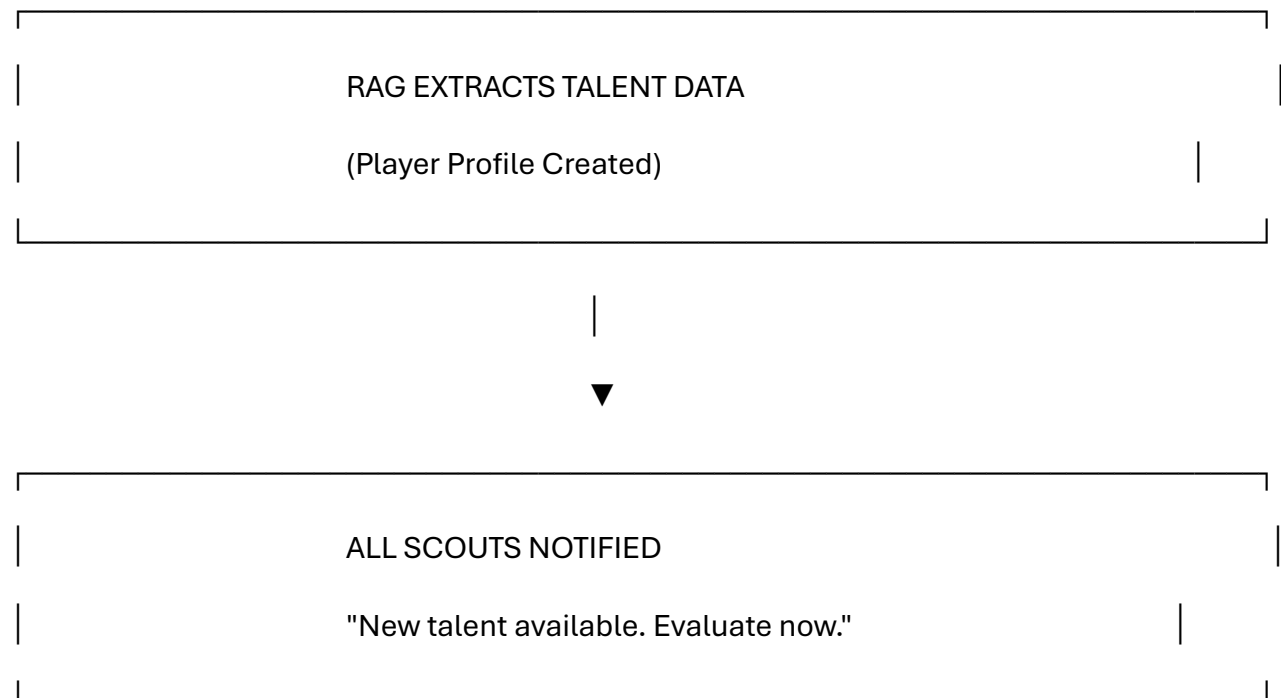
  "scoring_formula": {

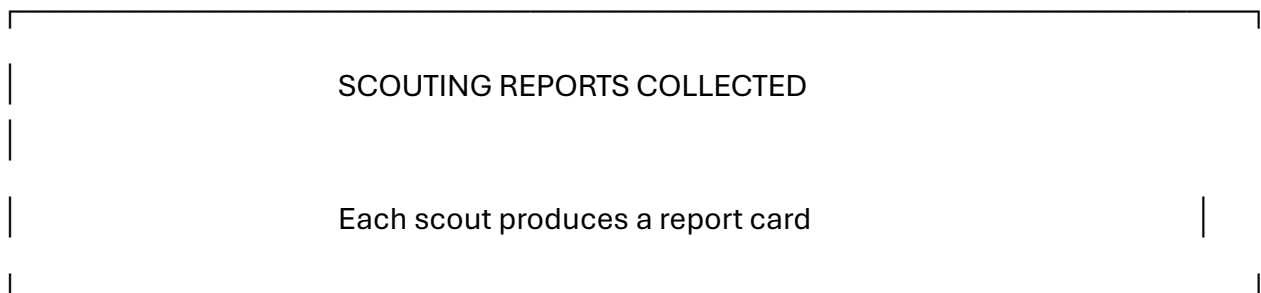
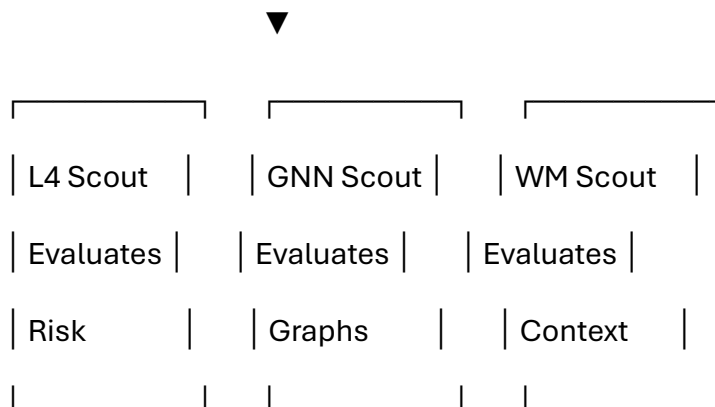
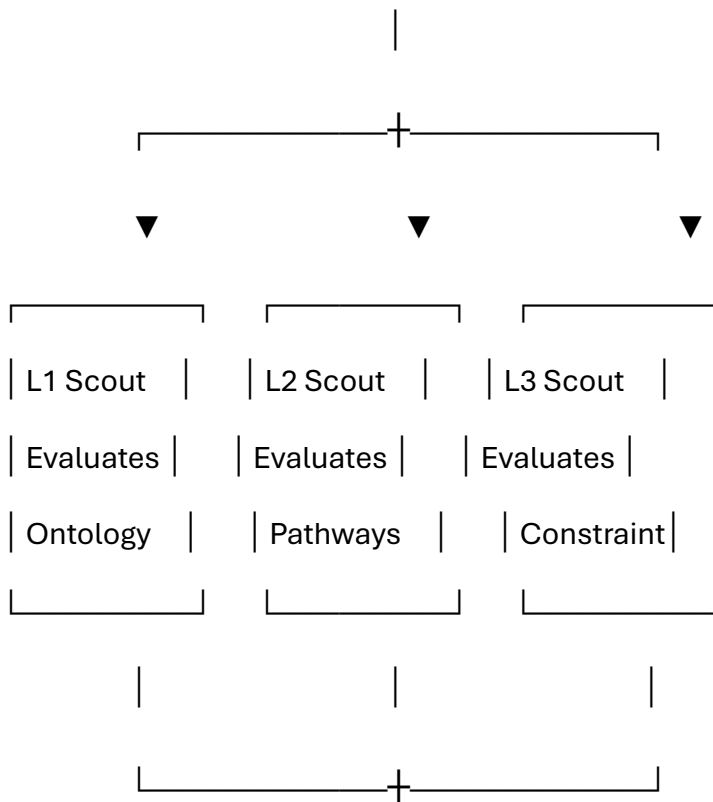
    "relevance": 0.35,
```

```
"confidence": 0.35,  
  
"past_performance": 0.20,  
  
"speed": 0.10  
  
,  
  
"past_record": {  
  
  "aerospace_materials": {"evaluations": 45, "accuracy": 0.89},  
  
  "automotive_parts": {"evaluations": 32, "accuracy": 0.85}  
  
},  
  
"average_evaluation_time_ms": 120,  
  
"scouting_report_template": "gnn_report_template.html"  
  
}
```

4. The Scouting Process (Campaign)

text





|



ORCHESTRATOR RANKS SCOUTS

Based on relevance, confidence, past record

|



HITL PRESENTS TOP SCOUTS

User (GM) reviews and decides

|



WINNING SCOUT'S REPORT ADOPTED

Or coalition of scouts' reports combined

5. Scout Report Card

Each scout produces a report card:

text

GNN SCOUT REPORT CARD

🇺🇸 RELEVANCE: 92%

"This data has clear relational structure. I am highly relevant."

🎯 CONFIDENCE: 87%

"Based on my analysis, I am confident in my graph predictions."

📈 PAST PERFORMANCE: 89%

"I have successfully evaluated 45 aerospace materials cases."

⚡ SPEED: 120ms (Top 2 fastest)

🔮 PREDICTION:

- Detected 3 anomalous data points
- Identified 5 key relationships
- Clustered into 2 distinct groups

🏆 OVERALL SCOUTING SCORE: 0.88

[\[View Full Report\]](#) [\[Accept\]](#) [\[Compare with Others\]](#)

6. Scout Ranking (The Draft Board)

Orchestrator ranks scouts by Scouting Score:

Rank	Scout	Relevance	Confidence	Past Perf	Speed	Total
1	GNN Scout	0.92	0.87	0.89	0.85	0.88
2	L4 Scout	0.89	0.91	0.82	0.70	0.85
3	WM Scout	0.85	0.88	0.85	0.75	0.84
4	L2 Scout	0.82	0.84	0.88	0.80	0.83
5	L1 Scout	0.88	0.79	0.83	0.90	0.82
6	CAUSAL Scout	0.80	0.86	0.80	0.65	0.79
7	L3 Scout	0.75	0.82	0.85	0.85	0.78
8	L5 Scout	0.70	0.80	0.90	0.60	0.75

7. Coalition Scouting (The All-Star Team)

Instead of a single scout, form a scouting coalition (all-star team):

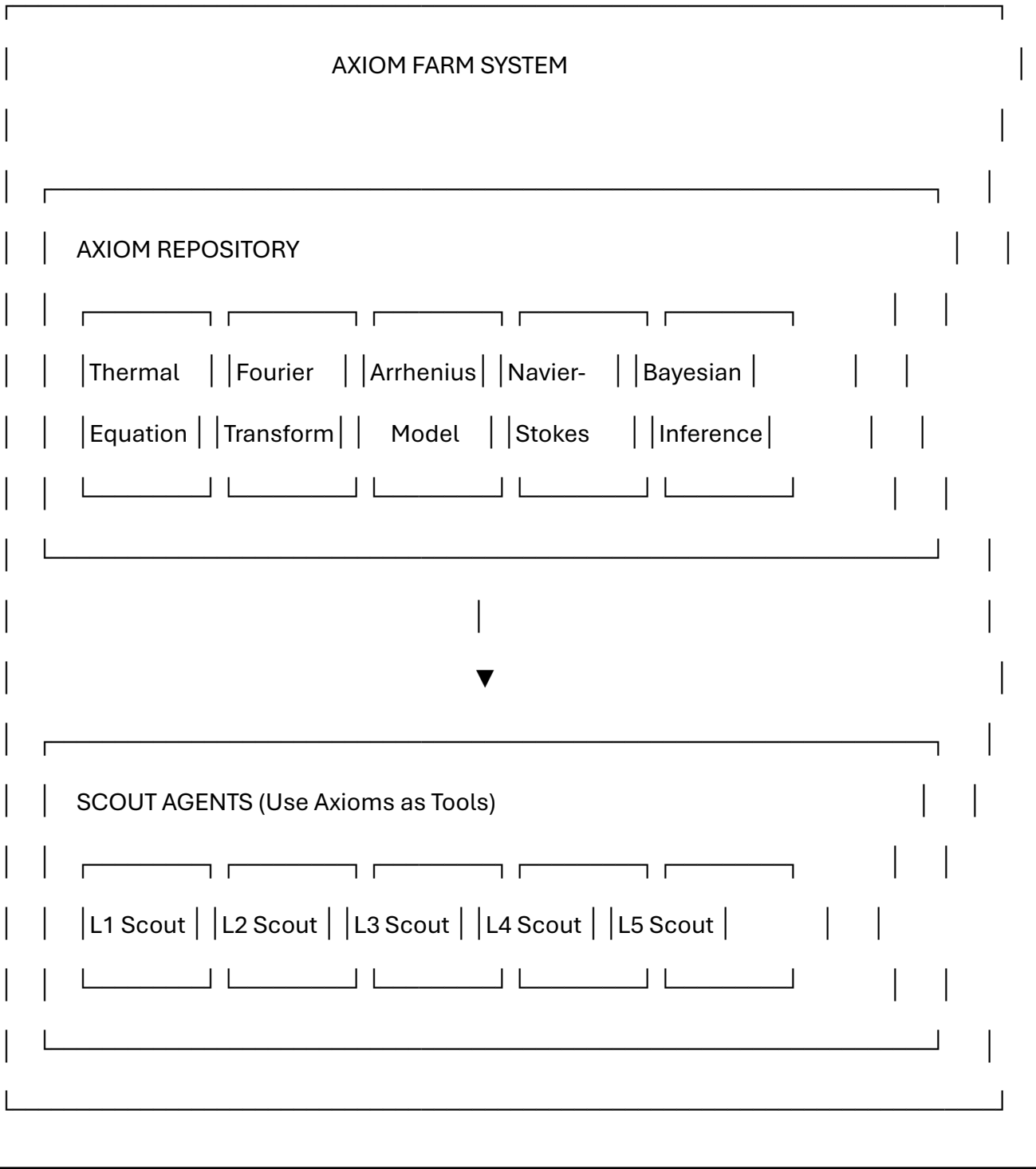
javascript

```
const coalition = {  
  "name": "Thermal Analysis All-Stars",  
  "members": ["L4 Scout", "GNN Scout", "WM Scout"],  
  "weighting": [0.4, 0.35, 0.25],  
  "combined_report": {  
    "risk_assessment": "Medium-High",  
    "graph_anomalies": 3,  
    "world_context": "Aerospace qualification required",  
    "recommendation": "Proceed with HALT testing"  
  },  
  "consensus_score": 0.86  
};
```

8. The Farm System (Axiom Repository)

Axioms (the "farm system") feed into scouts:

text



9. The HITL Scouting Combine (User Review)

Present top scouts to the user (General Manager):

text

SOVEREIGN SCOUTING COMBINE				
TALENT: SAAB_CFRP_001 (Aerospace Material)				
Overall Potential: 72% (Medium)				
TOP SCOUTS				
 GNN Scout (Score: 0.88)				
"Detected 3 anomalies, 5 relationships, 2 clusters"				
[Select]				
 L4 Scout (Score: 0.85)				
"Risk assessment: Medium-High. Confidence: 91%"				
[Select]				
 WM Scout (Score: 0.84)				
"Context: Aerospace qualification required"				
[Select]				

	👊 COALITION: Thermal All-Stars (Score: 0.86)	
	Members: L4 Scout, GNN Scout, WM Scout	
	[Select Coalition]	
	[Accept Top Scout] [Accept Coalition] [Manual Override]	

10. Scout Agent Implementation Architecture

javascript

// Base Scout Agent Class

```
class ScoutAgent {
  constructor(id, specialty, expertiseAreas, scoringFormula) {
    this.id = id;
    this.specialty = specialty;
    this.expertiseAreas = expertiseAreas;
    this.scoringFormula = scoringFormula;
    this.pastRecord = {};
    this.evaluationHistory = [];
  }
}
```

```
evaluate(talentData) {  
  
    const relevance = this.calculateRelevance(talentData);  
  
    const confidence = this.calculateConfidence(talentData);  
  
    const speed = this.measureSpeed();  
  
    const pastPerformance = this.getPastPerformance(talentData.domain);  
  
  
    const score = (relevance * this.scoringFormula.relevance) +  
                  (confidence * this.scoringFormula.confidence) +  
                  (pastPerformance * this.scoringFormula.past_performance) +  
                  ((1 - speed) * this.scoringFormula.speed);  
  
    return {  
  
        scoutId: this.id,  
  
        specialty: this.specialty,  
  
        relevance,  
  
        confidence,  
  
        speed,  
  
        pastPerformance,  
  
        score,  
  
        report: this.generateReport(talentData)  
  
    };  
}
```

```
generateReport(talentData) {
```

```
// Each scout implements its own report generation

    throw new Error("Must implement generateReport()");

}

}
```

// Specific Scout Implementations

```
class GNNScout extends ScoutAgent {

    generateReport(talentData) {

        return {

            anomalies: this.detectAnomalies(talentData),

            relationships: this.extractRelationships(talentData),

            clusters: this.performClustering(talentData),

            recommendation: this.makeRecommendation()

        };

    }

}
```

```
class L4RiskScout extends ScoutAgent {

    generateReport(talentData) {

        return {

            riskLevel: this.calculateRisk(talentData),

            confidenceInterval: this.computeConfidence(talentData),

            uncertainty: this.measureUncertainty(talentData),

            recommendation: this.makeRecommendation()

        };

    }

}
```

```
};  
  
}  
  
}
```

11. Agent Communication Protocol

Scouts communicate via event bus:

javascript

// Scout announces capability

```
EventBus.emit('scout:registered', {  
  scoutId: 'gnn_scout_v1',  
  specialty: 'Graph Neural Networks',  
  expertiseAreas: ['relationship_detection', 'anomaly_detection']  
});
```

// New talent available

```
EventBus.emit('talent:available', talentData);
```

// Scouts submit evaluations

```
EventBus.emit('scout:evaluation', evaluationResult);
```

// Orchestrator ranks scouts

```
EventBus.emit('orchestrator:ranking', rankedScouts);
```

// User selects winner


```
EventBus.emit('user:scout_selected', selectedScout);
```

12. Extensibility: Adding New Scouts

To add a new scout (e.g., "Quantum Scout"):

javascript

```
class QuantumScout extends ScoutAgent {  
  constructor() {  
    super('quantum_scout_v1', 'Quantum Computing',  
          ['quantum_simulation', 'optimization'],  
          { relevance: 0.4, confidence: 0.3, past_performance: 0.2, speed: 0.1 });  
  }  
  
  generateReport(talentData) {  
    // Quantum-specific evaluation logic  
    return {  
      quantum_speedup: this.estimateSpeedup(talentData),  
      qubit_requirements: this.calculateQubits(talentData),  
      recommendation: this.makeRecommendation()  
    };  
  }  
}  
  
// Register the new scout  
const quantumScout = new QuantumScout();
```

ScoutRegistry.register(quantumScout);

13. Comparison: Election vs. Scouting Metaphor

Aspect	Election Metaphor	Scouting Metaphor
Entities	Candidates	Scouts
Input	Voter preferences	Talent data
Output	Winner	Report card
User role	Voter	General Manager
Decision	Who wins	Who to sign
Multiple winners	Coalition government	All-star team
History	Term limits	Farm system

The scouting metaphor is richer for this domain because it emphasizes:

Evaluation not just competition

Development (farm system for axioms)

Team building (coalitions as all-star teams)

Long-term success (past performance matters)

14. Implementation Roadmap

Phase	Deliverable	Time
Phase 1	Scout agent base class + registry	2 days
Phase 2	L1~L5 scouts implementation	3 days
Phase 3	GNN, WM, CAUSAL scouts	2 days
Phase 4	Orchestrator + ranking system	2 days
Phase 5	HITL scouting combine UI	2 days
Phase 6	Coalition/all-star team formation	2 days
Phase 7	Farm system (axiom repository)	2 days
Phase 8	Event bus + communication	1 day
Total		16 days

✓ Natural competition (scouts compete to sign talent)

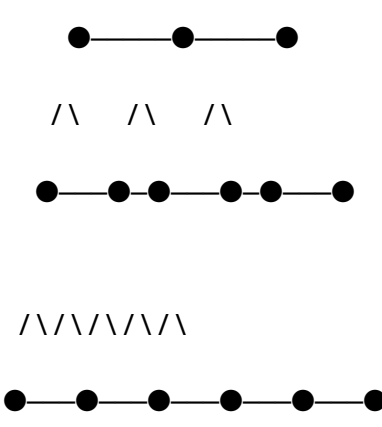
- ✓ Coalition building (all-star teams combine strengths)
- ✓ User-friendly metaphor (baseball scouting is intuitive)
- ✓ Extensibility (new scouts can be added)
- ✓ Audit trail (every scout's report is logged)
- ✓ Farm system (axioms develop into scouts)

The GNN Example (As You Described)

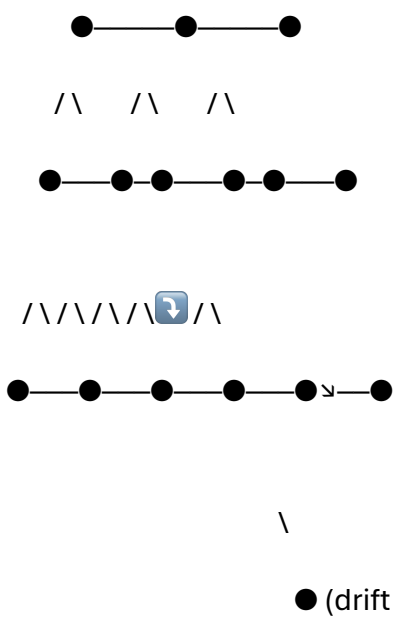
text

RAW DATA (from RAG)	
Time series, sensor readings, test results, anomalies	



GNN AGENT	
"I will build a 3D shape from these nodes and edges."	
3D SHAPE FORMATION	
	
"This is the ideal 3D shape from the axiom library."	

DEFORMATION DETECTION



"Suspicious drift detected at node 7. This could be a virus message or early sign of catastrophic failure."

TIME ROTATION

"Because the 3D shape is axis-rotatable, I can adjust the time dimension. Let me look BACKWARD to find the root cause..."

	[Time Rotated Backward]		
	"The drift originated at node 3, 47 cycles before failure."		
	[Time Rotated Forward]		
	"If drift continues, catastrophic failure will occur at node 12		
	within 23 cycles."		
	OUTPUT:		
	• Deformed 3D shape		
	• Drift detection (virus/catastrophe root)		
	• RCA (root cause analysis)		
	• Prediction (downtime event)		

The World Model Agent (Collaborating with GNN)

text

	WORLD MODEL AGENT		
	"I maintain the perfect 3D world model from all axioms."		

PERFECT WORLD MODEL

"This is how the system SHOULD behave under ideal conditions."

Thermal Model	Mechanical Model	Electrical Model
(Arrhenius)	(Fourier)	(Ohm's Law)

COMPARISON WITH GNN

GNN sends: "I found a drift at node 7."

World Model: "My perfect model has no drift at node 7.

The difference indicates a REAL anomaly,
not measurement noise."

Causal Matrix Agent: "Let me calculate the causal impact..."

OUTPUT:

• Confirmation that drift is real (not noise)	
• Difference map between ideal and actual	
• Trigger for Causal Matrix analysis	

The Causal Matrix Agent (Root Cause + Prediction)

text

CAUSAL MATRIX AGENT													
"I will calculate the causal relationships between every node and edge."													
CAUSAL MATRIX													
	N1	N2	N3	N4	N5	N6	N7	N8	N9	N10			
N1	0.0	0.8	0.2	0.0	0.0	0.0	0.0	0.0	0.0	0.0			
N2	0.0	0.0	0.9	0.1	0.0	0.0	0.0	0.0	0.0	0.0			
N3	0.0	0.0	0.0	0.7	0.3	0.0	0.0	0.0	0.0	0.0			
N4	0.0	0.0	0.0	0.0	0.6	0.4	0.0	0.0	0.0	0.0			
N5	0.0	0.0	0.0	0.0	0.0	0.5	0.5	0.0	0.0	0.0			
N6	0.0	0.0	0.0	0.0	0.0	0.0	0.4	0.6	0.0	0.0			
N7	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.3	0.7	0.0			

N8	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.2	0.8			
N9	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0			
N10	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0			
"The causal chain is clear: N1 → N2 → N3 → N4 → N5 → N6 → N7 (drift) → N8 → N9 → N10 (failure). The drift at N7 has 70% causal weight toward N8 and 30% toward N9. Failure at N10 is inevitable if drift continues."														
OUTPUT:														
• Causal matrix with weights														
• Root cause chain (N1 → N2 → ... → N10)														
• Probability of failure given current drift														

The L1~L5 Agents (Collaborative Pipeline)

L1 AGENT (Ontological)
"I classify this as: Aerospace Materials > Thermal Cycling > Reliability"
Confidence: 92%
"I pass this classification to L2."

|



L2 AGENT (Hypothesis)

"Based on L1's classification, I generate candidate pathways:

1. Fourier Transform (signal analysis)
2. Arrhenius Model (thermal aging)
3. GNN 3D Shape Analysis (drift detection)

I pass these to L3."

|



L3 AGENT (Admissibility)

"I check constraints:

- Fourier Transform: ☒ Admissible
- Arrhenius Model: ☒ Admissible
- GNN 3D Shape Analysis: ☒ Admissible

All three pathways pass. I pass them to L4."

|



L4 AGENT (Risk)

"I calculate risk for each pathway:

- Fourier Transform: Risk Score 0.23 (Low)

- Arrhenius Model: Risk Score 0.45 (Medium)

- GNN 3D Shape Analysis: Risk Score 0.12 (Very Low)

GNN has the lowest risk. I recommend it to L5."



L5 AGENT (Report)

"I generate the final report:

- GNN detected drift at node 7 (virus/catastrophe root)

- World Model confirmed drift is real

- Causal Matrix traced root cause to N1

- Prediction: Failure at N10 within 23 cycles

- Recommendation: Investigate N1-N3 chain immediately"

The Goldmine: How Agents Enrich Each Other

Agent	Gives to Others	Receives from Others
RAG	Raw data, initial confidence	Nothing (source)
GNN	3D shape, drift detection, time-rotated RCA	Perfect world model for comparison
World Model	Ideal 3D shape, multi-domain models	Drift location from GNN
Causal Matrix	Causal weights, root cause chain	Drift confirmation from World Model

L1	Ontological classification	None
L2	Candidate pathways	L1 classification
L3	Admissible pathways	L2 candidates
L4	Risk scores	L3 pathways
L5	Final report	All of the above

The Emergent Outcome

No single agent could produce the final result alone:

GNN alone: "I found a drift" (no context, no root cause, no prediction)

World Model alone: "Here is the ideal shape" (no drift detection)

Causal Matrix alone: "Here are weights" (no drift to apply them to)

Together: "Drift detected at node 7 (GNN). Confirmed real (World Model). Root cause at N1 (Causal Matrix). Failure predicted at N10 in 23 cycles (Causal Matrix). Recommendation: Investigate N1-N3 chain (L5)."

The Jade Design Principle

"The whole is greater than the sum of its parts."

Each agent is designed to be incomplete by itself. They must collaborate. They must share. They must enrich each other's data.

The measure of a good agent is not how well it competes, but how much it contributes to the collective intelligence.

Implementation Specification

javascript

```
class CooperativeAgent {
  constructor(id, specialty) {
    this.id = id;
```

```
this.specialty = specialty;

this.inputs = [];      // What data this agent needs from others

this.outputs = [];     // What data this agent produces for others

}
```

```
async process(inputs) {

  // Inputs from other agents

  const gnnOutput = inputs['gnn'];

  const worldModelOutput = inputs['world_model'];

  const causalOutput = inputs['causal_matrix'];


  // Agent-specific processing

  const myOutput = await this._compute(inputs);


  // Enrich other agents' outputs

  const enrichedData = this._enrich(myOutput, inputs);


  // Return both my output and enriched versions of others' outputs

  return {

    [this.id]: myOutput,

    ...enrichedData

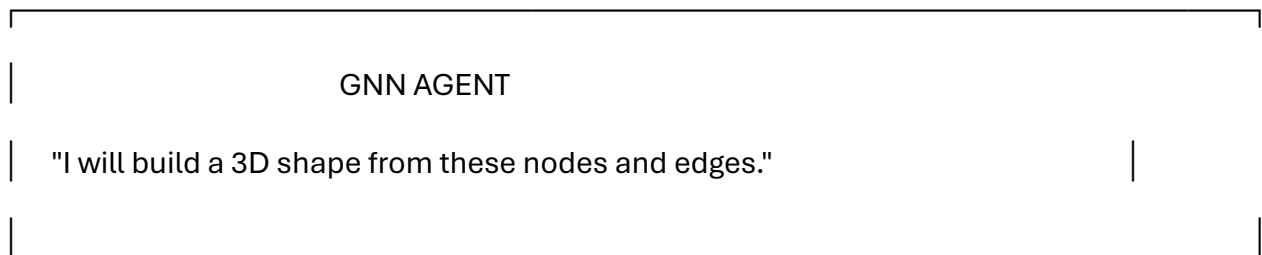
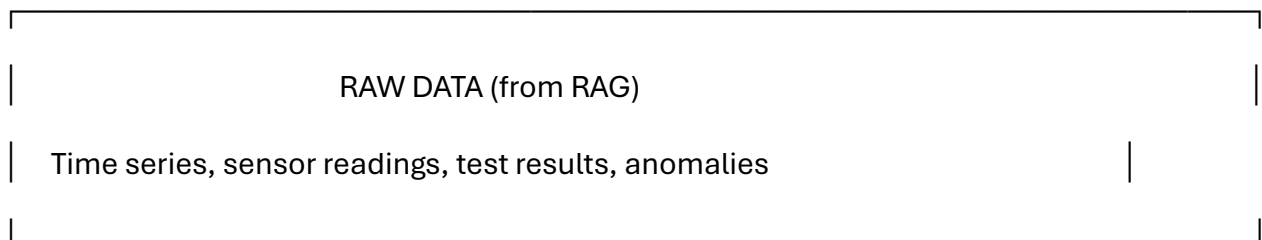
  };

}
```

```
_compute(inputs) {  
    // Each agent implements its own logic  
    throw new Error("Must implement _compute()");  
}  
  
_enrich(myOutput, inputs) {  
    // Each agent enriches others' data  
    return {};  
}  
}
```

The GNN Example (As You Described)

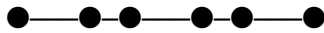
text



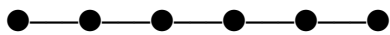
3D SHAPE FORMATION



/\ /\ /\



/\ /\ /\ /\ /\

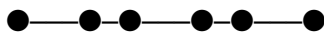


"This is the ideal 3D shape from the axiom library."

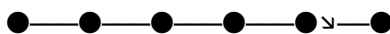
DEFORMATION DETECTION



/\ /\ /\



/\ /\ /\ /\ ↺ /\



\

● (drift detected)

"Suspicious drift detected at node 7. This could be a virus message or early sign of catastrophic failure."

TIME ROTATION

"Because the 3D shape is axis-rotatable, I can adjust the time dimension. Let me look BACKWARD to find the root cause..."

[Time Rotated Backward]

"The drift originated at node 3, 47 cycles before failure."

[Time Rotated Forward]

"If drift continues, catastrophic failure will occur at node 12 within 23 cycles."

OUTPUT:

- Deformed 3D shape
- Drift detection (virus/catastrophe root)

• RCA (root cause analysis)	
• Prediction (downtime event)	

The World Model Agent (Collaborating with GNN)

text

WORLD MODEL AGENT				
"I maintain the perfect 3D world model from all axioms."				
PERFECT WORLD MODEL				
"This is how the system SHOULD behave under ideal conditions."				
Thermal Model	Mechanical Model	Electrical Model		
(Arrhenius)	(Fourier)	(Ohm's Law)		
COMPARISON WITH GNN				

		GNN sends: "I found a drift at node 7."			
		World Model: "My perfect model has no drift at node 7."			
		The difference indicates a REAL anomaly,			
		not measurement noise."			
		Causal Matrix Agent: "Let me calculate the causal impact..."			

	OUTPUT:				
	• Confirmation that drift is real (not noise)				
	• Difference map between ideal and actual				
	• Trigger for Causal Matrix analysis				

The Causal Matrix Agent (Root Cause + Prediction)

text

	CAUSAL MATRIX AGENT				
	"I will calculate the causal relationships between every node and edge."				

CAUSAL MATRIX

[illegible]

"The causal chain is clear: N1 → N2 → N3 → N4 → N5 → N6 → N7 (drift)	
→ N8 → N9 → N10 (failure). The drift at N7 has 70% causal weight	
toward N8 and 30% toward N9. Failure at N10 is inevitable if drift	
continues."	

| OUTPUT: |

- Causal matrix with weights

• Root cause chain (N1 → N2 → ... → N10)	
• Probability of failure given current drift	

The L1~L5 Agents (Collaborative Pipeline)

text

L1 AGENT (Ontological)	
"I classify this as: Aerospace Materials > Thermal Cycling > Reliability"	
Confidence: 92%	
"I pass this classification to L2."	



L2 AGENT (Hypothesis)	
"Based on L1's classification, I generate candidate pathways:	
1. Fourier Transform (signal analysis)	
2. Arrhenius Model (thermal aging)	
3. GNN 3D Shape Analysis (drift detection)	
I pass these to L3."	



L3 AGENT (Admissibility)

"I check constraints:

- Fourier Transform:  Admissible
- Arrhenius Model:  Admissible
- GNN 3D Shape Analysis:  Admissible

All three pathways pass. I pass them to L4."



L4 AGENT (Risk)

"I calculate risk for each pathway:

- Fourier Transform: Risk Score 0.23 (Low)
- Arrhenius Model: Risk Score 0.45 (Medium)
- GNN 3D Shape Analysis: Risk Score 0.12 (Very Low)

GNN has the lowest risk. I recommend it to L5."



L5 AGENT (Report)

"I generate the final report:

- GNN detected drift at node 7 (virus/catastrophe root)

	- World Model confirmed drift is real	
	- Causal Matrix traced root cause to N1	
	- Prediction: Failure at N10 within 23 cycles	
	- Recommendation: Investigate N1-N3 chain immediately"	

The Goldmine: How Agents Enrich Each Other

Agent	Gives to Others	Receives from Others
RAG	Raw data, initial confidence	Nothing (source)
GNN	3D shape, drift detection, time-rotated RCA	Perfect world model for comparison
World Model	Ideal 3D shape, multi-domain models	Drift location from GNN
Causal Matrix	Causal weights, root cause chain	Drift confirmation from World Model
L1	Ontological classification	None
L2	Candidate pathways	L1 classification
L3	Admissible pathways	L2 candidates
L4	Risk scores	L3 pathways
L5	Final report	All of the above

The Emergent Outcome

No single agent could produce the final result alone:

GNN alone: "I found a drift" (no context, no root cause, no prediction)

World Model alone: "Here is the ideal shape" (no drift detection)

Causal Matrix alone: "Here are weights" (no drift to apply them to)

Together: "Drift detected at node 7 (GNN). Confirmed real (World Model). Root cause at N1 (Causal Matrix). Failure predicted at N10 in 23 cycles (Causal Matrix). Recommendation: Investigate N1-N3 chain (L5)."

The Jade Design Principle

"The whole is greater than the sum of its parts."

Each agent is designed to be incomplete by itself. They must collaborate. They must share.

They must enrich each other's data.

The measure of a good agent is not how well it competes, but how much it contributes to the collective intelligence.

Implementation Specification

javascript

```
class CooperativeAgent {  
  constructor(id, specialty) {  
    this.id = id;  
    this.specialty = specialty;  
    this.inputs = [];    // What data this agent needs from others  
    this.outputs = [];   // What data this agent produces for others  
  }  
  
  async process(inputs) {  
    // Inputs from other agents  
    const gnnOutput = inputs['gnn'];  
    const worldModelOutput = inputs['world_model'];  
    const causalOutput = inputs['causal_matrix'];  
  
    // Agent-specific processing  
    const myOutput = await this._compute(inputs);
```

```
// Enrich other agents' outputs

const enrichedData = this._enrich(myOutput, inputs);


// Return both my output and enriched versions of others' outputs
return {

  [this.id]: myOutput,

  ...enrichedData

};
}


_compute(inputs) {

  // Each agent implements its own logic

  throw new Error("Must implement _compute()");

}


_enrich(myOutput, inputs) {

  // Each agent enriches others' data

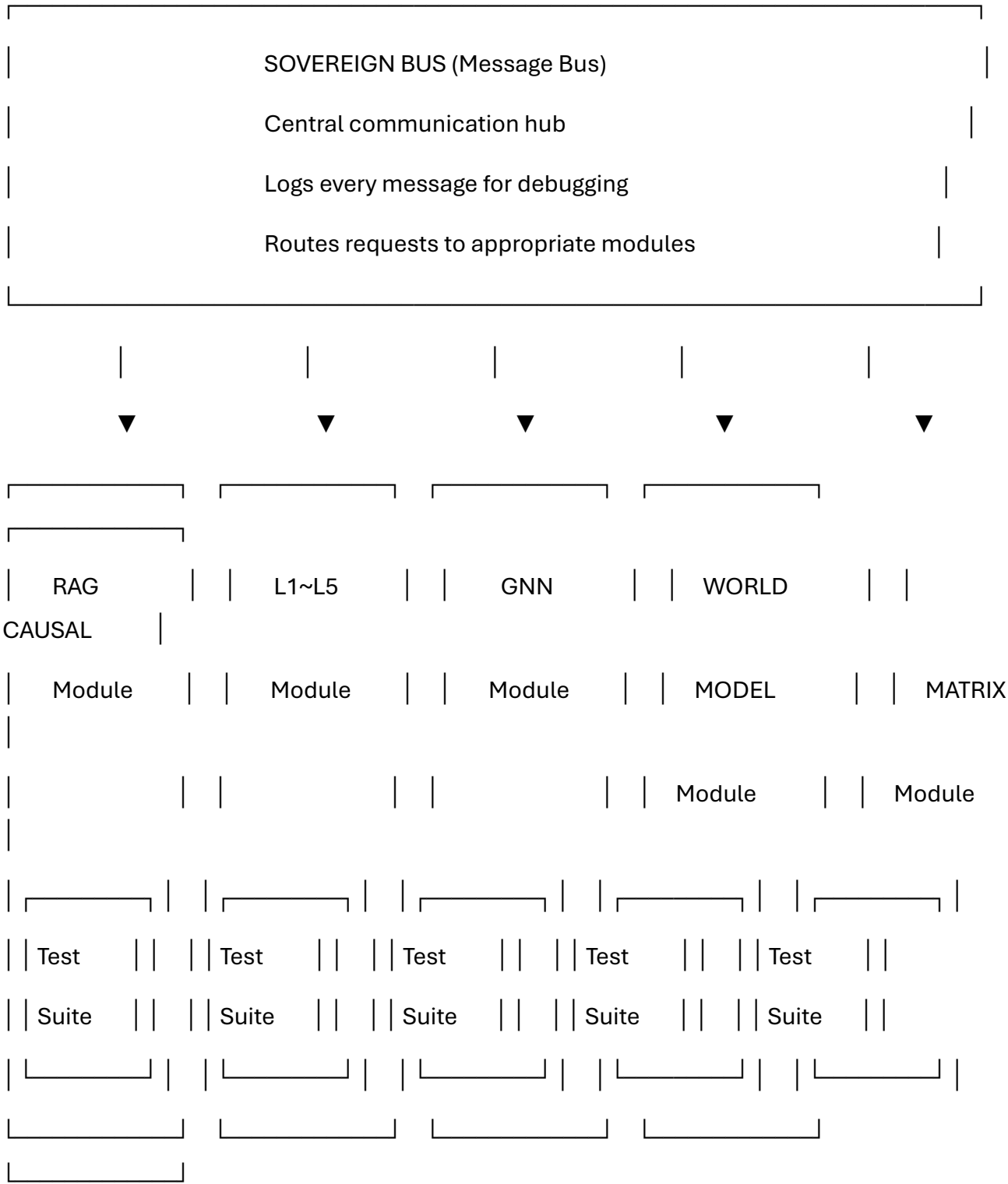
  return {};

}
}
```

The Solution: Module-Driven Architecture with MCP-like BUS

The Architecture

text



The BUS Protocol (MCP-like)

javascript

// Message format

```
{  
  "message_id": "uuid",  
  "timestamp": "2026-04-10T10:00:00Z",  
  "sender": "rag_module",  
  "recipient": "l1_module", // or "broadcast" for all  
  "message_type": "data_extracted",  
  "payload": {  
    "data": {...},  
    "confidence": 0.85  
  },  
  "trace_id": "for_debugging"  
}
```

Module Template

javascript

// modules/base_module.js

```
class SovereignModule {  
  constructor(name, bus) {  
    this.name = name;  
    this.bus = bus;  
    this.testSuite = new TestSuite(name);  
  }  
}
```

```
// Subscribe to messages

bus.subscribe(this.name, (message) => this.handle(message));

}

async handle(message) {

  this.bus.log(`[${this.name}] Received: ${message.message_type}`);

  try{

    const result = await this.process(message.payload);

    this.bus.send({

      sender: this.name,

      recipient: message.reply_to || "broadcast",

      message_type: `${message.message_type}_response`,

      payload: result,

      trace_id: message.trace_id

    });

  } catch (error) {

    this.bus.log(`[${this.name}] ERROR: ${error.message}`);

    this.bus.send({

      sender: this.name,

      recipient: message.sender,
```

```
        message_type: "error",

        payload: { error: error.message },

        trace_id: message.trace_id

    });

}

}
```

```
async process(payload) {

    // Implement in child class

    throw new Error("Must implement process()");

}
```

```
runTests() {

    return this.testSuite.run();

}

}
```

Example: RAG Module

javascript

// modules/rag_module.js

```
class RAGModule extends SovereignModule {

    constructor(bus) {

        super("rag_module", bus);
```

```
// Register tests

this.testSuite.add("PDF extraction", async () => {

    const result = await this.process({ file: "test.pdf" });

    return result.data !== null;

});

this.testSuite.add("CSV extraction", async () => {

    const result = await this.process({ file: "test.csv" });

    return result.data !== null;

});

this.testSuite.add("Gemini 3 Flash integration", async () => {

    const result = await this.process({ text: "Contract ID: 12345" });

    return result.domain === "contract";

});
}

async process(payload) {

    const { file, text } = payload;

    if (file) {

        return await this.extractFromFile(file);

    } else if (text) {

        return await this.extractFromText(text);

    }

}
```

```
}
```

```
throw new Error("No input provided");
```

```
}
```

```
async extractFromFile(file) {
```

```
  // Call Gemini 3 Flash
```

```
  const response = await fetch('/api/agent/seal/characterize', {
```

```
    method: 'POST',
```

```
    body: file
```

```
  });
```

```
  return await response.json();
```

```
}
```

```
async extractFromText(text) {
```

```
  // Direct Gemini call for testing
```

```
  const client = new genai.Client();
```

```
  const response = await client.models.generate_content({
```

```
    model: "gemini-3-flash-preview",
```

```
    contents: text
```

```
  });
```

```
  return JSON.parse(response.text);
```

```
}
```

```
}
```

Example: GNN Module

javascript

// modules/gnn_module.js

class GNNModule extends SovereignModule {

constructor(bus) {

super("gnn_module", bus);

this.testSuite.add("3D shape formation", async () => {

const result = await this.process({ nodes: 10, edges: 15 });

return result.shape_formed === true;

});

this.testSuite.add("Drift detection", async () => {

const result = await this.process({ nodes: 10, edges: 15, inject_drift: true });

return result.drift_detected === true;

});

this.testSuite.add("Time rotation (RCA)", async () => {

const result = await this.process({ nodes: 10, rotate_time: "backward" });

return result.root_cause !== null;

});

}

```
async process(payload) {

  const { nodes, edges, inject_drift, rotate_time } = payload;


  // Build 3D shape

  const shape = this.build3DShape(nodes, edges);


  // Detect drift

  const drift = this.detectDrift(shape, inject_drift);


  // Time rotation (RCA)

  let rootCause = null;

  if (rotate_time === "backward") {

    rootCause = this.findRootCause(shape, drift.location);

  } else if (rotate_time === "forward") {

    rootCause = this.predictFailure(shape, drift.location);

  }


  return {

    shape_formed: true,

    drift_detected: drift !== null,

    drift_location: drift?.location,

    root_cause: rootCause,

    recommendation: this.makeRecommendation(drift, rootCause)

  };
}
```



```
}
```

```
build3DShape(nodes, edges) { /* ... */ }
```

```
detectDrift(shape, inject_drift) { /* ... */ }
```

```
findRootCause(shape, location) { /* ... */ }
```

```
predictFailure(shape, location) { /* ... */ }
```

```
makeRecommendation(drift, rootCause) { /* ... */ }
```

```
}
```

The Sovereign BUS

javascript

// bus.js

```
class SovereignBUS {
```

```
  constructor() {
```

```
    this.subscribers = new Map();
```

```
    this.messageLog = [];
```

```
    this.traces = new Map();
```

```
  }
```

```
  subscribe(moduleName, callback) {
```

```
    if (!this.subscribers.has(moduleName)) {
```

```
      this.subscribers.set(moduleName, []);
```

```
    }
```

```
    this.subscribers.get(moduleName).push(callback);
```

```
}
```

```
send(message) {
```

```
    // Log every message
```

```
    this.messageLog.push(message);
```

```
    // Add to trace
```

```
    if (!this.traces.has(message.trace_id)) {
```

```
        this.traces.set(message.trace_id, []);
```

```
    }
```

```
    this.traces.get(message.trace_id).push(message);
```

```
    // Route to recipients
```

```
    if (message.recipient === "broadcast") {
```

```
        for (const [module, callbacks] of this.subscribers) {
```

```
            for (const cb of callbacks) {
```

```
                cb(message);
```

```
            }
```

```
        }
```

```
    } else {
```

```
        const callbacks = this.subscribers.get(message.recipient);
```

```
        if (callbacks) {
```

```
            for (const cb of callbacks) {
```

```
                cb(message);
```

```
    }  
  }  
}  
}
```

```
log(message) {  
  console.log(` [BUS] ${message}` );  
}
```

```
getTrace(trace_id) {  
  return this.traces.get(trace_id);  
}
```

```
getMessagesByModule(moduleName) {  
  return this.messageLog.filter(m => m.sender === moduleName || m.recipient ===  
moduleName);  
}
```

```
clear() {  
  this.messageLog = [];  
  this.traces.clear();  
}  
}
```

Testing a Module in Isolation

javascript

```
// test_rag_module.js
```

```
import { SovereignBUS } from './bus.js';
```

```
import { RAGModule } from './modules/rag_module.js';
```

```
async function testRAGModule() {
```

```
    const bus = new SovereignBUS();
```

```
    const rag = new RAGModule(bus);
```

```
    // Subscribe to bus to see responses
```

```
    bus.subscribe("test", (message) => {
```

```
        console.log(`[TEST] Received: ${message.message_type}`);
```

```
    });
```

```
    // Send test message
```

```
    bus.send({
```

```
        sender: "test",
```

```
        recipient: "rag_module",
```

```
        message_type: "extract",
```

```
        payload: { text: "Contract ID: 12345" },
```

```
        trace_id: "test_001",
```

```
        reply_to: "test"
```

```
    });
```

```
// Wait for response

await new Promise(resolve => setTimeout(resolve, 2000));


// Check trace

const trace = bus.getTrace("test_001");

console.log("Trace:", JSON.stringify(trace, null, 2));


// Run module's test suite

const results = await rag.runTests();

console.log("Test results:", results);
}

testRAGModule();
```

Debugging with Trace

When something fails, you have a complete trace:

json

```
{
  "trace_id": "user_session_001",
  "messages": [
    {"sender": "ui", "recipient": "rag_module", "message_type": "upload_file", "timestamp":
"10:00:00"},
    {"sender": "rag_module", "recipient": "l1_module", "message_type": "data_extracted",
"timestamp": "10:00:02"},
```

```

    {"sender": "l1_module", "recipient": "gnn_module", "message_type": "classified_data",
"timestamp": "10:00:03"},

    {"sender": "gnn_module", "recipient": "l5_module", "message_type": "drift_detected",
"timestamp": "10:00:05"},

    {"sender": "l5_module", "recipient": "ui", "message_type": "report_ready", "timestamp":
"10:00:07"}

]

}

```

Benefits of Module-Driven + BUS Architecture

Problem	Solution
RAG bug affects everything	RAG module fails, others continue
Hard to test	Each module has its own test suite
Can't find failure point	Trace shows exact message where failure occurred
Tight coupling	Modules only know the BUS, not each other
Long debug cycles	Isolate and test individual modules
RAG integration issues	Test RAG module in isolation first

Implementation Roadmap

Phase	Deliverable	Time
Phase 1	Sovereign BUS implementation	2 days
Phase 2	Base module class + test framework	2 days
Phase 3	RAG module + test suite	2 days
Phase 4	L1~L5 modules + test suites	3 days
Phase 5	GNN, WM, CAUSAL modules + test suites	3 days
Phase 6	Integration + end-to-end tests	2 days
Phase 7	Debugging tools (trace viewer)	2 days
Total		16 days

Aspect	Human Spec	AI Spec
Language	Natural language, prose	Structured, deterministic, schema-based
Ambiguity	Tolerated ("approximately", "should")	Zero tolerance ("exactly", "must")
Format	Word/PDF document	JSON / YAML / TypeScript interfaces
Examples	Illustrative	Executable test cases
Dependencies	Described	Explicitly imported

The AI-Ready Spec Format

yaml

```
# spec/module_rag.yaml
```

```
version: "1.0.0"
```

```
module: rag
```

```
description: "Extract structured data from files using Gemini 3 Flash"
```

```
# No gray areas - everything is deterministic
```

```
input_schema:
```

```
  $schema: "https://json-schema.org/draft/07/schema"
```

```
  type: object
```

```
  required: [source]
```

```
  properties:
```

```
    source:
```

```
      oneOf:
```

```
        - type: object
```

```
          required: [file_path]
```

properties:

file_path:

type: string

pattern: "^.*\\.\\.(pdf|csv|json)\$"

- type: object

required: [text]

properties:

text:

type: string

minLength: 1

maxLength: 10000

output_schema:

type: object

required: [domain, nodes, assessment]

properties:

domain:

type: string

enum: [contract, financial, technical, aerospace, general]

nodes:

type: array

minItems: 1

maxItems: 100

items:

type: object

required: [id, name, value, confidence]

properties:

id:

type: string

pattern: "[A-Z0-9_]+\$"

name:

type: string

minLength: 1

maxLength: 100

value:

type: string

confidence:

type: number

minimum: 0

maximum: 1

multipleOf: 0.01

assessment:

type: object

required: [conf, tier, reason]

properties:

conf:

type: number

minimum: 0

maximum: 1

multipleOf: 0.01

tier:

type: integer

enum: [1, 2, 3]

reason:

type: string

minLength: 10

maxLength: 500

Deterministic test points (not examples)

test_points:

- id: "rag_001"

name: "PDF contract extraction"

input:

file_path: "fixtures/contract.pdf"

expected_output:

domain: "contract"

nodes.0.id: "CONTRACT_ID"

nodes.0.confidence: { min: 0.85, max: 1.0 }

assessment.tier: 1

deterministic_check: "output.nodes.length >= 3"

- id: "rag_002"

name: "CSV financial extraction"

input:

file_path: "fixtures/financial.csv"

expected_output:

domain: "financial"

nodes.0.name: "revenue"

deterministic_check: "output.assessment.conf > 0.8"

- id: "rag_003"

name: "Gemini 3 Flash integration"

input:

text: "Contract ID: 12345, Amount: \$25000, Date: 2024-01-15"

expected_output:

domain: "contract"

nodes: { length: 3 }

deterministic_check: "output.nodes[0].id == 'CONTRACT_ID'"

Question 2: How Can Gray Areas Be Proactively Addressed with Deterministic Approaches?

Gray areas are eliminated by design, not discovered by testing.

Gray Area Sources and Deterministic Solutions

Gray Area Source	Deterministic Solution
"Extract relevant fields"	Explicit field list in spec
"Handle errors gracefully"	Defined error codes + fallback behavior
"Process the data"	State machine with defined transitions
"Use reasonable confidence"	Fixed thresholds (0.85 = high, 0.60 = medium)
"May take some time"	Hard timeout (30 seconds)

"Return appropriate response"	Exact schema with all possible variants
-------------------------------	---

Example: Eliminating "Extract Relevant Fields" Gray Area

yaml

NOT acceptable (gray)

extract_fields: "Extract relevant fields from the document"

ACCEPTABLE (deterministic)

extract_fields:

required: [contract_id, amount, date]

optional: [party_name, description, terms]

reject_if_missing_required: true

confidence_threshold: 0.85

Example: Eliminating "Handle Errors" Gray Area

yaml

NOT acceptable (gray)

error_handling: "Handle errors gracefully"

ACCEPTABLE (deterministic)

error_handling:

error_codes:

- code: "E001"

condition: "file not found"

action: "return {error: 'File not found', fallback: null}"

- code: "E002"

condition: "Gemini API timeout"

action: "retry with fallback extraction after 30s"

- code: "E003"

condition: "invalid JSON response"

action: "retry up to 3 times, then return default extraction"

default_action: "return {error: 'Unknown error', fallback: null}"

log_all_errors: true

Question 3: Deterministic Test Points for Testability, Serviceability, Reliability

The Three Pillars

Pillar	Definition	Implementation
Testability	Can each component be tested in isolation?	Module test suites with mock dependencies
Serviceability	Can failures be diagnosed quickly?	Trace IDs, structured logs, health endpoints
Reliability	Does it work consistently?	Idempotent operations, retries, timeouts

Deterministic Test Point Specification

yaml

test_point:

id: "gnn_010"

name: "3D shape drift detection"

type: "deterministic"

Setup

setup:

- action: "load_fixture"

fixture: "perfect_shape_10_nodes.json"

- action: "inject_drift"

at_node: 7

drift_magnitude: 0.15

Input

input:

nodes: "\$fixture.nodes"

edges: "\$fixture.edges"

inject_drift: true

Expected output (exact, not approximate)

expected:

shape_formed: true

drift_detected: true

drift_location: "N7"

confidence: { operator: ">=", value: 0.80 }

Performance requirement

performance:

max_duration_ms: 500

max_memory_mb: 256

Serviceability check

serviceability:

logs_contains: "Drift detected at N7"

trace_contains: "gnn_module -> causal_module"

Reliability check

reliability:

run_count: 100

expected_success_rate: 0.99

Question 4: Is There an Off-the-Shelf Design Methodology?

Yes. Multiple open-source methodologies exist.

Methodology 1: DDD (Domain-Driven Design)

Source	Description
Eric Evans' "Domain-Driven Design"	Core principles for modeling complex domains
Open source: ddd-by-examples	Practical examples
awesome-ddd	Curated resources

Methodology 2: Event Storming

Source	Description
Alberto Brandolini's Event Storming	Collaborative modeling for complex systems
Open source: event-storming-guide	Free templates and examples

Methodology 3: C4 Model for Architecture

Source	Description
Simon Brown's C4 Model	Context, Containers, Components, Code
Open source: c4-model	Diagrams as code

Methodology 4: BDD (Behavior-Driven Development)

Source	Description
Cucumber/Gherkin	Given-When-Then format
Open source: cucumber	Executable specifications

Methodology 5: OpenAPI / JSON Schema

Source	Description
OpenAPI Specification	API contract specification
JSON Schema	Data structure validation

The Combined Methodology for AI Tools

yaml

Combined approach using open-source standards

methodology:

- name: "OpenAPI 3.0"
 - purpose: "API contract specification"
 - format: "YAML"
 - example: "spec/openapi.yaml"
- name: "JSON Schema"
 - purpose: "Data structure validation"
 - format: "JSON/YAML"
 - example: "spec/schemas/"
- name: "Gherkin (Cucumber)"
 - purpose: "Behavior specifications"

format: "Feature files"

example: "spec/features/rag.feature"

- name: "C4 Model"

purpose: "Architecture diagrams"

format: "PlantUML/Mermaid"

example: "docs/architecture/c4/"

- name: "DDD Bounded Contexts"

purpose: "Module boundaries"

format: "Markdown + diagrams"

example: "docs/domain/"

The Complete AI-Ready Specification Structure

text

spec/

├─ openapi.yaml # API contracts

├─ schemas/

| └─ rag_input.json # JSON Schema

| └─ rag_output.json

| └─ gnn_input.json

```
|   └─ gnn_output.json
|
|   └─ ...
|
└─ features/

|   └─ rag.feature           # Gherkin BDD specs
|
|   └─ gnn.feature
|
|   └─ l1_l5.feature
|
|   └─ ...
|
└─ test_points/

|   └─ rag_test_points.yaml  # Deterministic test points
|
|   └─ gnn_test_points.yaml
|
|   └─ ...
|
└─ architecture/

|   └─ context.puml          # C4 context diagram
|
|   └─ containers.puml       # C4 containers
|
|   └─ components.puml      # C4 components
└─ README.md                 # Spec index
```

Example: Gherkin Spec for RAG Module

gherkin

spec/features/rag.feature

Feature: RAG Module - Document Extraction

Background:

Given the Gemini 3 Flash API is available

And the API key is configured

Scenario: Extract contract data from PDF

When I upload a file "contract.pdf"

Then the response domain should be "contract"

And the nodes should include "CONTRACT_ID", "AMOUNT", "DATE"

And each node should have confidence ≥ 0.85

And the assessment tier should be 1

Scenario: Handle corrupted PDF

When I upload a file "corrupted.pdf"

Then the response should include an error

And the error code should be "E002"

And the system should fall back to default extraction

Scenario: Gemini API timeout

Given the Gemini API will respond after 35 seconds

When I upload a file "contract.pdf"

Then the response should be received within 30 seconds

And the system should use fallback extraction

The Naming Convention Standard (No Ambiguity)

Rule 1: Model Names (Exact, Verified)

Incorrect (Gray)	Correct (Deterministic)
gemini-3-flash	gemini-3-flash-preview
gemini-3.0-flash	gemini-3-flash-preview
gemini-3.1-flash-lite	gemini-3.1-flash-lite-preview
gemini-pro	gemini-1.5-pro
gemini-2.0	gemini-2.0-flash-exp

Rule: Use exact model strings as published in official documentation. No abbreviations. No approximations.

Rule 2: Naming Case Convention

Element	Convention	Example
Module names	lowercase_with_underscores	rag_module, gnn_module
Class names	PascalCase	RAGModule, GNNAgent
Function/Method names	camelCase	extractData(), detectDrift()
Constants	UPPER_SNAKE_CASE	GEMINI_3_FLASH_PREVIEW, MAX_RETRY_COUNT
Environment variables	UPPER_SNAKE_CASE	GEMINI_API_KEY, LOG_LEVEL
API endpoints	lowercase_with_underscores	/api/agent/seal/characterize
JSON field names	snake_case	contract_id, confidence_score
File names	lowercase_with_underscores	rag_module.py, test_gnn.py
Test IDs	MODULE_NUMBER_DESCRIPTION	RAG_001_pdf_extraction
Error codes	LETTER_NUMBER	E001_file_not_found, T001_timeout
Trace IDs	UUID or TIMESTAMP_RANDOM	20260410_083000_a3f2

Rule 3: ID Naming Patterns

ID Type	Pattern	Example
Node ID	N{number}	N1, N2, N7
Edge ID	E{number}	E1, E2
Axiom ID	{domain}_{type}_{number}	CONTRACT_D_001, THERMAL_B_002
Test Point ID	{MODULE}_{NUMBER}_{description}	RAG_001_pdf_extraction
Trace ID	{timestamp}_{random}	20260410_083000_a3f2
Session ID	SESSION_{timestamp}_{user}	SESSION_20260410_083000_leo

Rule 4: Dataset Naming

Dataset Type	Pattern	Example
Fixture file	{domain}_{type}_{number}.{ext}	contract_pdf_001.pdf
Test input	input_{module}_{test_id}.json	input_rag_001.json
Expected output	expected_{module}_{test_id}.json	expected_rag_001.json
Log file	log_{module}_{date}.log	log_rag_20260410.log

Rule 5: Action Names (No Ambiguity)

Action	Deterministic Name	Description
Extract data	extract_structured_data()	Returns JSON schema
Detect drift	detect_3d_shape_drift()	Returns drift location
Rotate time	rotate_time_dimension(direction)	direction: FORWARD or BACKWARD
Find root cause	trace_root_cause(start_node)	Returns node chain
Predict failure	predict_failure_timeline()	Returns cycles to failure
Confirm HITL	confirm_human_in_loop(decision)	decision: ACCEPT, REJECT, MODIFY

Rule 6: Enum Values (Uppercase, No Spaces)

Enum Type	Values
Domain	CONTRACT, FINANCIAL, TECHNICAL, AEROSPACE, GENERAL
Tier	TIER_1, TIER_2, TIER_3
Direction	FORWARD, BACKWARD, NONE
Status	PENDING, IN_PROGRESS, COMPLETED, FAILED
Confidence	HIGH, MEDIUM, LOW

The Specification Header (Every Spec Must Include)

yaml

spec_header.yaml

version: "1.0.0"

last_updated: "2026-04-10"

naming_convention:

modules: "lowercase_with_underscores"

classes: "PascalCase"

functions: "camelCase"

constants: "UPPER_SNAKE_CASE"

env_vars: "UPPER_SNAKE_CASE"

api_endpoints: "lowercase_with_underscores"

json_fields: "snake_case"

files: "lowercase_with_underscores"

test_ids: "MODULE_NUMBER_DESCRIPTION"

error_codes: "LETTER_NUMBER"

model_names:

gemini: "gemini-3-flash-preview"

embedding: "text-embedding-004"

embedding_dimension: 768

error_codes:

E001: "File not found"

E002: "Gemini API timeout"

E003: "Invalid JSON response"

E004: "Schema validation failed"

E005: "Module timeout"

T001: "Test assertion failed"

trace_id_format: "{timestamp}_{random_8}"

timestamp_format: "YYYYMMDD_HHMMSS"

The "Gray Area Elimination" Checklist

Before writing any spec, verify:

#	Check	Status
1	Every model name is exact (from official docs)	☐
2	Every naming convention is defined	☐
3	No abbreviations allowed	☐
4	Every enum has defined values	☐
5	Every action has a deterministic name	☐
6	Every error has a code	☐
7	Every ID follows a pattern	☐
8	No "etc.", "and so on", "similar to"	☐
9	No "approximately", "around", "about"	☐
10	No "may", "might", "could", "should"	☐

Example: Good vs Bad Spec

 BAD (Gray Area)

text

The system will extract data from the uploaded file using Gemini.

It will detect any anomalies and report them.

 GOOD (Deterministic)

yaml

action: extract_structured_data

input:

file:

path: string

content: base64

model: "gemini-3-flash-preview"

output:

domain: enum[CONTRACT, FINANCIAL, TECHNICAL, AEROSPACE, GENERAL]

nodes: array of Node objects

assessment:

conf: float (0.0 to 1.0)

tier: enum[TIER_1, TIER_2, TIER_3]

reason: string

anomaly_detection:

method: "3d_shape_drift_detection"

confidence_threshold: 0.85

output:

drift_detected: boolean

drift_location: string or null

confidence: float

The Naming Convention Standard (No Ambiguity)

Rule 1: Model Names (Exact, Verified)

Incorrect (Gray)	Correct (Deterministic)
gemini-3-flash	gemini-3-flash-preview
gemini-3.0-flash	gemini-3-flash-preview
gemini-3.1-flash-lite	gemini-3.1-flash-lite-preview
gemini-pro	gemini-1.5-pro
gemini-2.0	gemini-2.0-flash-exp

Rule: Use exact model strings as published in official documentation. No abbreviations. No approximations.

Rule 2: Naming Case Convention

Element	Convention	Example
Module names	lowercase_with_underscores	rag_module, gnn_module
Class names	PascalCase	RAGModule, GNNAgent
Function/Method names	camelCase	extractData(), detectDrift()
Constants	UPPER_SNAKE_CASE	GEMINI_3_FLASH_PREVIEW, MAX_RETRY_COUNT
Environment variables	UPPER_SNAKE_CASE	GEMINI_API_KEY, LOG_LEVEL
API endpoints	lowercase_with_underscores	/api/agent/seal/characterize
JSON field names	snake_case	contract_id, confidence_score
File names	lowercase_with_underscores	rag_module.py, test_gnn.py
Test IDs	MODULE_NUMBER_DESCRIPTION	RAG_001_pdf_extraction
Error codes	LETTER_NUMBER	E001_file_not_found, T001_timeout
Trace IDs	UUID or TIMESTAMP_RANDOM	20260410_083000_a3f2

Rule 3: ID Naming Patterns

ID Type	Pattern	Example
Node ID	N{number}	N1, N2, N7
Edge ID	E{number}	E1, E2
Axiom ID	{domain}_{type}_{number}	CONTRACT_D_001, THERMAL_B_002
Test Point ID	{MODULE}_{NUMBER}_{description}	RAG_001_pdf_extraction
Trace ID	{timestamp}_{random}	20260410_083000_a3f2
Session ID	SESSION_{timestamp}_{user}	SESSION_20260410_083000_leo

Rule 4: Dataset Naming

Dataset Type	Pattern	Example
Fixture file	{domain}_{type}_{number}.{ext}	contract_pdf_001.pdf
Test input	input_{module}_{test_id}.json	input_rag_001.json
Expected output	expected_{module}_{test_id}.json	expected_rag_001.json
Log file	log_{module}_{date}.log	log_rag_20260410.log

Rule 5: Action Names (No Ambiguity)

Action	Deterministic Name	Description
Extract data	extract_structured_data()	Returns JSON schema
Detect drift	detect_3d_shape_drift()	Returns drift location
Rotate time	rotate_time_dimension(direction)	direction: FORWARD or BACKWARD
Find root cause	trace_root_cause(start_node)	Returns node chain
Predict failure	predict_failure_timeline()	Returns cycles to failure
Confirm HITL	confirm_human_in_loop(decision)	decision: ACCEPT, REJECT, MODIFY

Rule 6: Enum Values (Uppercase, No Spaces)

Enum Type	Values
Domain	CONTRACT, FINANCIAL, TECHNICAL, AEROSPACE, GENERAL
Tier	TIER_1, TIER_2, TIER_3
Direction	FORWARD, BACKWARD, NONE
Status	PENDING, IN_PROGRESS, COMPLETED, FAILED
Confidence	HIGH, MEDIUM, LOW

The Specification Header (Every Spec Must Include)

yaml

spec_header.yaml

version: "1.0.0"

last_updated: "2026-04-10"

naming_convention:

modules: "lowercase_with_underscores"

classes: "PascalCase"

functions: "camelCase"

constants: "UPPER_SNAKE_CASE"

env_vars: "UPPER_SNAKE_CASE"

api_endpoints: "lowercase_with_underscores"

json_fields: "snake_case"

files: "lowercase_with_underscores"

test_ids: "MODULE_NUMBER_DESCRIPTION"

error_codes: "LETTER_NUMBER"

model_names:

gemini: "gemini-3-flash-preview"

embedding: "text-embedding-004"

embedding_dimension: 768

error_codes:

E001: "File not found"

E002: "Gemini API timeout"

E003: "Invalid JSON response"

E004: "Schema validation failed"

E005: "Module timeout"

T001: "Test assertion failed"

trace_id_format: "{timestamp}_{random_8}"

timestamp_format: "YYYYMMDD_HHMMSS"

The "Gray Area Elimination" Checklist

Before writing any spec, verify:

#	Check	Status
1	Every model name is exact (from official docs)	☐
2	Every naming convention is defined	☐
3	No abbreviations allowed	☐
4	Every enum has defined values	☐
5	Every action has a deterministic name	☐
6	Every error has a code	☐
7	Every ID follows a pattern	☐
8	No "etc.", "and so on", "similar to"	☐
9	No "approximately", "around", "about"	☐
10	No "may", "might", "could", "should"	☐

Example: Good vs Bad Spec

 BAD (Gray Area)

text

The system will extract data from the uploaded file using Gemini.

It will detect any anomalies and report them.

 GOOD (Deterministic)

yaml

action: extract_structured_data

input:

file:

path: string

content: base64

model: "gemini-3-flash-preview"

output:

domain: enum[CONTRACT, FINANCIAL, TECHNICAL, AEROSPACE, GENERAL]

nodes: array of Node objects

assessment:

conf: float (0.0 to 1.0)

tier: enum[TIER_1, TIER_2, TIER_3]

reason: string

anomaly_detection:

method: "3d_shape_drift_detection"

confidence_threshold: 0.85

output:

drift_detected: boolean

drift_location: string or null

confidence: float
